

(Use Simulator – CPU Sim 3.6.9 or any higher version for the implementation)

1. Create a machine based on the following architecture:

Registers

IR DR AC AR PC I E

16 bits 16 bits 16 bits 12 bits 12 bits 1 bit 1 bit

Basic Computer Instructions

Memory Reference Register Reference

Symbol Hex Symbol Hex

AND 0xxx

Direct

Addressing

CLA 7800

ADD 1xxx CLE 7400

LDA 2xxx CMA 7200

STA 3xxx CME 7100

BUN 4xxx CIR 7080

BSA 5xxx CIL 7040

ISZ 6xxx INC 7020

AND_I 8xxx

Indirect

Addressing

SPA 7010

ADD_I 9xxx SNA 7008

LDA_I Axxx SZA 7004

STA_I Bxxx SZE 7002

BUN_I Cxxx HLT 7001

BSA_I Dxxx INP F800

ISZ_I Exxx OUT F400

Opcode Address

Solution:

To implement the specified machine architecture using **CPU Sim 3.6.9** (or higher) on Ubuntu, follow these steps to install the software and configure the hardware and instruction set.

Phase 1: Installation on Ubuntu

CPU Sim is a Java application and requires a Java Runtime Environment (JRE) to run.

[Ask Ubuntu +1](#)

1. **Install Java:** Open your terminal and run `sudo apt-get install openjdk-11-jre` (or any JRE version 1.5 or later).
2. **Download CPU Sim:** Get the package from the [CPU Sim Home Page](#).

3. **Run the Simulator:** Unzip the file, navigate to the folder in the terminal, and execute:
`java -cp CPUSim3.9.jar:jhall.jar cpusim.Main` (Replacing "3.9" with your specific version number).

Colby Computer Science +2

Phase 2: Create the Machine Architecture

In the CPU Sim interface, go to **Modify** -> **Hardware Modules** to define your components.

Scribd +1

1. Define Registers

Create the following registers with their specified bit widths:

Scribd +1

- **IR (Instruction Register):** 16 bits
- **DR (Data Register):** 16 bits
- **AC (Accumulator):** 16 bits
- **AR (Address Register):** 12 bits
- **PC (Program Counter):** 12 bits
- **I (Addressing Mode bit):** 1 bit
- **E (Extended Carry bit):** 1 bit

2. Define Memory (RAM)

- **Name:** Main Memory
- **Length:** 4096 (since AR is 12 bits,

).

- **Cell Size:** 16 bits.

Scribd +1

Phase 3: Implementing Instructions

Go to **Modify** -> **Machine Instructions** to add the instruction set.

[Scribd +1](#)

1. Instruction Formats

Create fields to match the 16-bit instruction format:

- **Opcode:** 4 bits (Bits 12–15).
- **Address:** 12 bits (Bits 0–11).

[Scribd +1](#)

2. Memory Reference Instructions (MRI)

Add these instructions with their hex opcodes. For indirect addressing (e.g., **AND_I**), the 15th bit is set to 1.

[Scribd +3](#)

- **AND / AND_I:** Hex 0xxx / 8xxx
- **ADD / ADD_I:** Hex 1xxx / 9xxx
- **LDA / LDA_I:** Hex 2xxx / Axxx
- **STA / STA_I:** Hex 3xxx / Bxxx
- **BUN / BUN_I:** Hex 4xxx / Cxxx
- **BSA / BSA_I:** Hex 5xxx / Dxxx
- **ISZ / ISZ_I:** Hex 6xxx / Exxx

3. Register Reference & I/O Instructions

These use the full 16-bit hex code as they do not require an address field:

[GitHub +2](#)

- **CLA:** 7800 (Clear AC)
- **CLE:** 7400 (Clear E)
- **HLT:** 7001 (Halt)
- **INP:** F800 (Input)
- **OUT:** F400 (Output)

Phase 4: Fetch and Execute Cycles

1. **Fetch Sequence:** Define the microinstructions in **Modify** -> **Fetch Sequence**.

- $AR \leftarrow PC$
- $IR \leftarrow M[AR]$
- $PC \leftarrow PC + 1$
- Decode IR.

Execute Sequence: For each instruction (like ADD), define the specific microinstructions (e.g., $DR \leftarrow M[AR]$, then $AC \leftarrow AC + DR$).

2. Create a Fetch routine of the instruction cycle.

In a typical **CPU Simulator** (like **CPUSim**), creating a fetch routine involves defining a sequence of microinstructions that retrieve an instruction from memory and prepare it for decoding.

Colby Computer Science +1

Fetch Routine Steps

A standard fetch routine follows these sequential microinstructions to move data through the registers:

GeeksforGeeks +3

1. **Transfer PC to AR:** Copy the address of the next instruction from the **Program Counter (PC)** to the **Address Register (AR)**.
2. **Memory Read:** Retrieve the instruction stored at the address in the **AR** from memory (RAM) and place it into the **Instruction Register (IR)** (or sometimes a Data Register first).
3. **Increment PC:** Add 1 to the **PC** so it points to the next memory address for the next cycle.
4. **Decode:** Signal the Control Unit to interpret the operation code (opcode) now stored in the **IR**.

Indian Institute of Technology (IIT) Roorkee +5

Implementation in CPUSim (Ubuntu)

If you are using the popular Java-based **CPUSim** on Ubuntu, follow these steps to set up the routine:

Scribd

- **Define Registers:** Go to Modify -> Hardware Modules and ensure you have PC, AR (Address Register), and IR (Instruction Register) defined.
- **Create Microinstructions:**
 - o **TransferRtoR:** Create a transfer named PC->AR with PC as source and AR as destination.
 - o **MemoryAccess:** Create a "Read" operation named M[AR] ->IR using Main memory, IR as data, and AR as address.

- o **Increment:** Create an increment for PC by 1.
- o **Decode:** Add a Decode microinstruction for the IR register.
- **Assemble the Sequence:** Open the Fetch Sequence editor and drag-and-drop your new microinstructions in this exact order:
 - PC $\rightarrow$ AR
 - M[AR] $\rightarrow$ IR
 - PC $\rightarrow$ PC + 1
 - DECODE IR
 - END

Scribd +2

Once saved, the simulator will automatically run this sequence at the start of every instruction cycle.

3. Write an assembly program to simulate ADD operation on two user-entered numbers.

To simulate an ADD operation on two user-entered numbers using a **CPU Simulator** in Ubuntu, you typically follow a two-step process: defining the machine instructions (if using a microcode-level simulator like CPUSim) and then writing the assembly code.

Scribd +1

1. Define Machine Instructions (For CPUSim)

If you are using [CPUSim](#), you must first ensure your "machine" has instructions for reading input and performing addition:

Scribd +1

- **read**: An instruction to get user input from the console and place it into a register (e.g., DR or R0).
- **add**: A machine instruction that takes values from two registers (or a register and memory), performs the arithmetic, and stores the result.
- **write**: An instruction to display the result from a register to the console.
- **stop**: A command to terminate execution.

YouTube +4

2. Assembly Program Template

Once the machine instructions are defined, you can write the assembly code in the simulator's text editor:

```
assembly
; Program to add two user-entered numbers
read          ; Prompts user for the first number (stored in DR/R0)
transfertoDR  ; (If needed) Moves first number to a temporary register/memory
read          ; Prompts user for the second number (stored in DR/R0)
add           ; Adds the two values and stores the sum in a register
write         ; Displays the sum on the console
stop          ; Ends the program
Use code with caution.
```

Scribd +1

Alternative: Using NASM on Ubuntu (System Level)

If you prefer to run assembly directly on Ubuntu rather than in a specific educational simulator, you can use the Netwide Assembler (NASM):

Ask Ubuntu +1

1. **Install NASM:** Run `sudo apt install nasm` in your terminal.
2. **Write the Code:** Create a file named `add.asm`.
3. **Basic Logic:**
 1. Use `sys_read` (syscall 3) to get input from the keyboard.
 2. Convert the input (ASCII string) to an integer.
 3. Perform the addition using the `add` instruction (e.g., `add eax, ebx`).
 4. Convert the result back to ASCII to display it using `sys_write` (syscall 4).

Ask Ubuntu +4

Running the Simulator

To run a Java-based simulator like CPUSim on Ubuntu, ensure you have the Java Runtime Environment (JRE 1.8+) installed, then execute the `.jar` file using:

```
java -jar CPUSim.jar
```

4. Write an assembly program to simulate SUBTRACT operation on two user-entered numbers.

To simulate a **SUBTRACT** operation in a CPU Simulator on Ubuntu, you can use **CPU Sim 3.4** or the **CPU-OS Simulator**. These simulators allow you to write assembly code, compile it into machine code, and observe register changes step-by-step.

Colby Computer Science +4

1. Setup CPU Simulator on Ubuntu

If you are using **CPU Sim 3.4**, you can launch it from the terminal since it is a Java-based application:

- **Navigate** to your simulator folder.
- **Run** the following command:
`java -cp CPUSim3.4.jar:jhall.jar:CPUSimHelp3.4.jar cpusim.Main.`

Colby Computer Science

2. Assembly Program (General Logic)

Most simulators follow a structure where you load user input into registers and then perform the subtraction. Below is a generic assembly structure commonly used in these environments:

```
assembly
; Program to subtract two user-entered numbers
INP          ; Get first number from user input
STA NUM1     ; Store it in memory address 'NUM1'
INP          ; Get second number from user input
STA NUM2     ; Store it in memory address 'NUM2'

LDA NUM1     ; Load first number back into Accumulator
SUB NUM2     ; Subtract second number from Accumulator
OUT          ; Output the result to the console
HLT          ; Halt/stop the program

NUM1 DAT 0   ; Storage for first number
NUM2 DAT 0   ; Storage for second number
Use code with caution.
```

3. Detailed Steps to Execute

1. **Open Editor:** Paste the code above into the simulator's **Editor window**.

2. **Compile & Load:** Use the **Compile and Load (F5)** button to translate the assembly into machine code and load it into RAM.
3. **Run Program:** Click the **Run** or **Play** button. The simulator will prompt you twice for input (e.g., enter 10 then 4).
4. **Observe Results:**
 1. Watch the **Accumulator (AC)** or **Registers** update as each line executes.
 2. The final difference (e.g., 6) will appear in the **Output Console**.

Debug: Use the **Step** button to execute instructions one-by-one and track how the value moves from the input device to the register.

YouTube +5

For more advanced simulators that model specific hardware like the **8085**, you would use instructions like LDA to load data from a memory location and SUB to perform the arithmetic.

5. Write an assembly program to simulate the following logical operations on two user entered numbers.

- i. AND
- ii. OR
- iii. NOT
- iv. XOR
- v. NOR
- vi. NAND

Solution:

To simulate logical operations in a CPU Simulator on Ubuntu, you typically use a Java-based tool like **CPUSim**. This simulator allows you to define your own instruction set and write assembly code to execute microinstructions.

Colby Computer Science +4

Below is a general assembly program structure for logical operations. Note that basic simulators often lack direct NOR or NAND instructions, so these are typically simulated using NOT after an OR or AND operation.

Assembly Program for Logical Operations

This program assumes a standard register-based architecture (like R0, R1, etc.) and uses pseudo-code common in [CPU Sim instruction sets](#).

assembly

```
; Logical Operations Simulation Program
; Input: User enters two numbers into registers R1 and R2

START:  INP R1          ; Read first number into R1
        INP R2          ; Read second number into R2

; i. AND Operation
        AND R3, R1, R2  ; R3 = R1 AND R2
        OUT R3          ; Display AND result

; ii. OR Operation
        OR R4, R1, R2   ; R4 = R1 OR R2
        OUT R4          ; Display OR result

; iii. NOT Operation
        NOT R5, R1      ; R5 = NOT R1 (inverts first number)
```

```

        OUT R5          ; Display NOT result

; iv. XOR Operation
        XOR R6, R1, R2  ; R6 = R1 XOR R2
        OUT R6          ; Display XOR result

; v. NOR Operation (OR followed by NOT)
        OR R7, R1, R2   ; R7 = R1 OR R2
        NOT R7, R7      ; R7 = NOT (R1 OR R2) -> NOR
        OUT R7          ; Display NOR result

; vi. NAND Operation (AND followed by NOT)
        AND R8, R1, R2  ; R8 = R1 AND R2
        NOT R8, R8      ; R8 = NOT (R1 AND R2) -> NAND
        OUT R8          ; Display NAND result

        HLT             ; Stop the machine

```

Use code with caution.

How to Run on Ubuntu

1. **Install Java:** Ensure you have the Java Runtime Environment installed by running `sudo apt install default-jre` in your terminal.
2. **Download CPUSim:** Download the CPUSim package (e.g., [CPUSim 3.4](#)).
3. **Launch the Simulator:** Navigate to the folder in your terminal and run:
`java -cp CPUSim3.4.jar:jhall.jar:CPUSimHelp3.4.jar cpusim.Main.`
4. **Assemble and Load:**
 1. Open the text editor in the simulator and paste the assembly code.
 2. Go to **Execute > Assemble & Load**.

Execute: Click **Run** or **Step** to see the values change in the registers.

6. Write an assembly program for simulating following memory-reference instructions.

- i. ADD
- ii. LDA
- iii. STA
- iv. BUN
- v. ISZ

Solution:

To simulate these memory-reference instructions in Ubuntu using **CPU-Sim** (or a similar basic computer simulator), you first need to define the hardware and machine instructions in the software. Memory-reference instructions typically use a 16-bit format: 1 bit for addressing mode (I), 3 bits for the opcode, and 12 bits for the address.

PVPSIT +2

1. Setting up the Simulator (Pre-requisite)

Before writing the program, ensure your simulator has the following hardware modules and "Fetch" routine:

Scribd +1

- **Registers:** AC (Accumulator), PC (Program Counter), AR (Address Register), IR (Instruction Register), and DR (Data Register).
- **Memory:** A RAM module (e.g., 4096 words of 16 bits each).
- **Fetch Sequence:** Typically includes steps like $AR \leftarrow PC$, $IR \leftarrow M[AR]$, $PC \leftarrow PC + 1$, and Decode IR.

GitHub +1

2. Assembly Program Example

The following program demonstrates each instruction by performing simple data manipulation and branching.

assembly

```
; --- Program Start ---
ORG 000          ; Start the program at address 0
    LDA X        ; Load value of X into AC
    ADD Y        ; Add value of Y to AC (AC = X + Y)
```

```

    STA RES      ; Store result of (X + Y) into memory location RES
    ISZ CTR      ; Increment CTR; if result is 0, skip the next instruction
    BUN END      ; Branch Unconditionally to END (skipped if ISZ condition met)
    LDA Y        ; (Optional) This would run if ISZ CTR = 0
END, HLT        ; Stop the program

; --- Data Section ---
X,   DEC 10      ; Variable X with value 10
Y,   DEC 5       ; Variable Y with value 5
RES, DEC 0       ; Storage for the result
CTR, DEC -1      ; Counter for ISZ (will become 0 after one increment)
END
Use code with caution.

```

3. Instruction Descriptions

In your simulator, each instruction must be defined with its specific micro-operations:

Scribd +2

- **LDA (Load to AC):** Transfers a word from memory to the Accumulator.
 - o *Micro-ops:* DR <- M[AR], AC <- DR.
- **ADD (Add to AC):** Adds the memory word to the content of the AC and stores the sum in AC.
 - o *Micro-ops:* DR <- M[AR], AC <- AC + DR.
- **STA (Store AC):** Transfers the content of the AC into a memory word.
 - o *Micro-ops:* M[AR] <- AC.
- **BUN (Branch Unconditionally):** Transfers control to the instruction at the specified address by updating the PC.
 - o *Micro-ops:* PC <- AR.
- **ISZ (Increment and Skip if Zero):** Increments the memory word; if the result is 0, the next instruction is skipped by incrementing the PC again.
 - o *Micro-ops:* DR <- M[AR], DR <- DR + 1, M[AR] <- DR, If (DR = 0) then PC <- PC + 1.

Scribd +4

4. Running on Ubuntu

1. **Install Java:** Most CPU Simulators (like CPU-Sim 3.6.9) require a Java Runtime Environment (JRE). You can install it via terminal: `sudo apt install default-jre`.
2. **Launch Simulator:** Run the `.jar` file of your simulator.
3. **Define Machine:** Use the "Modify" menu to set up your Registers, RAM, and Machine Instructions (Opcode values).
4. **Execute:** Paste the assembly code into the text editor, assemble it, and use "Step" or "Run" to observe the register changes.

7. Write an assembly language program to simulate the machine for following register reference instructions and determine the contents of AC, E, PC, AR and IR registers in decimal after the execution:

- i. CLA
- ii. CMA
- iii. CME
- iv. HLT

Solution:

To simulate these register-reference instructions in Ubuntu, you can use the **CPU Sim** application, a Java-based tool commonly used in computer architecture courses.

Colby Computer Science +2

Assembly Language Program

The following program executes the four instructions in sequence. In this simulator's architecture, these are typically 16-bit instructions with a hex code range starting with 7 (register-reference).

Scribd +1

```
assembly
; Program to simulate register-reference instructions
CLA    ; Clear Accumulator (AC <- 0)
CMA    ; Complement Accumulator (AC <- AC')
CME    ; Complement Extended bit (E <- E')
HLT    ; Halt the simulation
Use code with caution.
```

Determining Register Contents

Assuming the program starts at memory address 0 (PC = 0) and the **Accumulator (AC)** and **E (flip-flop)** are initially 0.

Instruction	Step	AC (Decimal)	E	PC (Decimal)	AR (Decimal)	IR (Hex/Dec)
Initial State	-	0	0	0	0	0
CLA	Clear AC	0	0	1	0	7800 / 30720
CMA	Complement AC	-1 (or 65535)*	0	2	1	7200 / 29184
CME	Complement E	-1 (or 65535)*	1	3	2	7100 / 28928

HLT	Halt	-1 (or 65535)*	1	4	3	7001 / 28673
-----	------	----------------	---	---	---	--------------

**Note: In a 16-bit two's complement system, complementing a 0 value (0000...) results in all ones (1111...), which is decimal -1 or 65,535 depending on the simulator's display mode.*

Register Explanations

- **AC (Accumulator):** Stores the result of logic/arithmetic operations. CLA clears it, and CMA flips all bits.
- **E (Extended Bit):** A 1-bit register used for carry/overflow. CME toggles its value from 0 to 1 or vice versa.
- **PC (Program Counter):** Tracks the address of the next instruction. It increments by 1 after each fetch cycle.
- **AR (Address Register):** Holds the memory address currently being accessed. During the fetch cycle of each instruction, it equals the current PC.
- **IR (Instruction Register):** Holds the binary/hex code of the instruction currently being executed.

8. Write an assembly language program to simulate the machine for following register reference instructions and determine the contents of AC, E, PC, AR and IR registers in decimal after the execution:

- i. INC
- ii. SPA
- iii. SNA
- iv. SZE

Solutions:

This program is designed for a basic CPU simulator (commonly used in computer architecture courses, such as CPUSim) that simulates a 16-bit machine with an Accumulator (AC), End-carry (E), Program Counter (PC), Address Register (AR), and Instruction Register (IR).

1. Assembly Language Program (`register_ref.a`)

This program demonstrates **INC** (Increment AC), **SPA** (Skip if Positive), **SNA** (Skip if Negative), and **SZE** (Skip if Zero) instructions, along with supporting instructions to set up the register states.

[GitHub](#) +3

assembly

```
; Register Reference Instructions Simulation
; Assumes standard Mano Machine architecture
; Registers: AC (16-bit), E (1-bit), PC (12-bit)

ORG 100          ; Program starts at memory address 100

; --- Setup initial state ---
CLA              ; Clear AC
CME              ; Complement E (Set E=1)
CMA              ; Complement AC (AC = FFFF)
INC              ; Increment AC (AC = 0000, E becomes 1 due to carry)

; --- Instruction 1: INC ---
; Assuming AC is 0005, E is 0
; LDA CONST_5
; INC            ; AC becomes 6
```

```

; --- Instruction 2: SPA (Skip if Positive) ---
; If AC > 0 (MSB is 0), PC = PC + 1
CLA
INC
INC          ; AC = 2 (Positive)
SPA          ; Should skip next instruction
HLT          ; If SPA fails, it stops here

; --- Instruction 3: SNA (Skip if Negative) ---
; If AC < 0 (MSB is 1), PC = PC + 1
CMA          ; AC = FFFF (Negative)
SNA          ; Should skip next instruction
HLT          ; If SNA fails, it stops here

; --- Instruction 4: SZE (Skip if Zero E-carry) ---
; If E = 0, PC = PC + 1
CLE          ; E = 0
SZE          ; Should skip next instruction
HLT          ; If SZE fails, it stops here

HLT          ; End of program

; Data
CONST_5: DEC 5

```

2. Execution and Register Contents (Decimal)

To determine the contents in decimal, we follow the execution flow assuming a standard 16-bit 2's complement simulator where FFFF is -1 and 0000 is 0.

Note: After any HLT (Halt) instruction, the PC points to the next memory address.

Assuming the program finishes at the final HLT:

Register	Description	Typical Content (Hex)	Final Content (Decimal)
AC	Accumulator	0000 (after SZE)	0
E	End Carry Bit	0 (after CLE)	0
PC	Program	107 (example)	263 (Address of next

	Counter		instruction)
AR	Address Register	106	262 (Address of last HLT)
IR	Instruction Reg	7001 (SZE)	28673 (Machine code for SZE)

3. How to run on Ubuntu

1. **Install JRE:** `sudo apt update && sudo apt install default-jre`
2. **Download CPUSim:** Download the CPUSim.jar file from its official site.
3. **Run:** `java -jar CPUSim.jar`
4. **Assemble and Load:** Create a new machine, load the assembly code, assemble, and step through to see register changes.

9. Write an assembly language program to simulate the machine for following register reference instructions and determine the contents of AC, E, PC, AR and IR registers in decimal after the execution:

- i. CIR
- ii. CIL

Solutions:

To simulate the **CIR (Circulate Right)** and **CIL (Circulate Left)** register-reference instructions in a CPU Simulator (like **CPUSim** or the **Mano Machine Simulator**) on Ubuntu, you can use the following assembly program. These instructions shift the contents of the Accumulator (**AC**) and the Extended bit (**E**) in a circular fashion.

Scribd +4

Assembly Language Program

assembly

```
/ Program to simulate CIR and CIL
/ Initialize AC and E before shifts
/ Assuming program starts at address 0

ORG 0
LDA VAL      / Load a value into AC (e.g., 2 in decimal)
CLE          / Clear E flip-flop (E = 0)
CIR          / Circulate AC and E right
CIL          / Circulate AC and E left
HLT          / Halt the program

VAL, DEC 2   / Value used for testing
END
Use code with caution.
```

Determining Register Contents (Decimal)

The exact decimal values depend on the initial state of the registers. Below is the step-by-step trace assuming the program starts at address 0 and VAL is at address 5 with an initial decimal value of 2 (binary 0000 0000 0000 0010 for a 16-bit AC).

Scribd +2

1. After CIR (Circulate Right AC and E)

In a circulate right operation, the least significant bit (LSB) of AC goes into E, and the old value of E moves into the most significant bit (MSB) of AC.

- **Initial state:** AC = 2 (0000 . . . 0010), E = 0.

- **Execution:** The LSB of AC (0) moves to E. The 0 from E moves to the MSB of AC. The remaining bits shift right.
- **Register Contents:**
 - o **AC: 1** (Binary 0000 . . . 0001)
 - o **E: 0**
 - o **PC: 4** (Points to the next instruction, CIL, at address 4)
 - o **AR: 5** (Address of VAL from previous LDA or address of current instruction)
 - o **IR: 28672** (Hex 7080, the decimal equivalent of the CIR instruction code)

GeeksforGeeks +3

2. After CIL (Circulate Left AC and E)

Following the CIR, we now shift left. The MSB of AC goes to E, and the old value of E moves to the LSB of AC.

- **Initial state (from above):** AC = 1 (0000 . . . 0001), E = 0.
- **Execution:** The MSB of AC (0) moves to E. The 0 from E moves to the LSB of AC. The remaining bits shift left.
- **Register Contents:**
 - o **AC: 2** (Binary 0000 . . . 0010)
 - o **E: 0**
 - o **PC: 5** (Points to HLT at address 5)
 - o **AR: 4** (Last accessed instruction address)
 - o **IR: 28676** (Hex 7040, the decimal equivalent of the CIL instruction code)

Scribd +1

How to Run in Ubuntu

1. **Install Java:** Most CPU Simulators (like [CPU Sim](#)) are Java-based and platform-independent.

```
bash
```

```
sudo apt update  
sudo apt install default-jre
```

Use code with caution.

2. **Launch Simulator:** Open your terminal, navigate to the folder containing your .jar file, and run:

```
bash
```

```
java -jar CPUSim.jar
```

Use code with caution.

3. **Execute:** Paste the assembly code above into the text editor, **Assemble** it, and use the **Step** button to watch the decimal contents change in the register pane.

10. Write an assembly program that reads in integers and adds them together; until a negative non-zero number is read in. Then it outputs the sum (not including the last number).

Solution:

To write this program for the **CPU Simulator** (commonly the Wombat or similar educational architectures used in Ubuntu environments), you must initialize a register to hold the running sum and use a loop that breaks when a negative value is detected.

YouTube +2

Assembly Program Logic

The following code uses standard CPU Simulator syntax (like Wombat1) where READ takes user input into the accumulator (AC), ADD performs addition, and JUMP controls the loop.

Kalamazoo College +1

assembly

```
; Initialize sum to 0
LOAD zero
STORE sum

loop:
    READ          ; Read integer from keyboard into AC
    JNEG done     ; If AC < 0, jump to output (don't add last number)

    ADD sum       ; Add current sum to the new input in AC
    STORE sum     ; Store the updated result back in 'sum'
    JUMP loop     ; Repeat the process

done:
    LOAD sum      ; Load final sum into AC
    WRITE        ; Output the sum to the console
    STOP         ; Terminate program

; Data declarations
sum: .data 0
zero: .data 0
Use code with caution.
```

Key Components

- **Registers:** In many simulators, the **Accumulator (AC)** is the primary register used for both reading input and performing arithmetic.
- **Conditional Jump:** The JNEG (Jump if Negative) instruction is critical here; it checks if the last number entered was less than zero to stop the addition process before that number is added.
- **Ubuntu Setup:** You can run the [CPU Sim 3.4 JAR](#) on Ubuntu using the terminal with the command: `java -jar CPUSim3.4.jar`

11. Write an assembly program that reads in integers and adds them together; until zero is read in. Then it outputs the sum.

Solution:

To write an assembly program that reads integers and sums them until zero is entered using a **CPU Simulator** (like the common CPU OS Simulator), you can use a loop structure with a conditional jump.

Medium +2

Assembly Program Logic

In most common CPU simulators, the registers (e.g., R00, R01) are used to store the running total and the user input.

assembly

```
; Initialize R00 (the sum) to 0
MOV #0, R00

LOOP:
    ; Read an integer from the user into R01
    IN R01

    ; Check if the input is zero
    CMP #0, R01

    ; If input is zero, jump to the end
    JZ DONE

    ; Add the input to the running sum in R00
    ADD R01, R00

    ; Repeat the loop
    JMP LOOP

DONE:
    ; Output the final sum stored in R00
    OUT R00

    ; Terminate the program
    HALT
```

Use code with caution.

Steps to Run in Ubuntu

If you are using a simulator like **CPU Sim** or a local Linux-based environment:

1. **Open your CPU Simulator:** You can often find browser-based versions or Java-based versions (.jar) that run on Ubuntu using the command `java -jar CPUSim.jar`.
2. **Enter the Code:** Copy the assembly instructions into the simulator's code editor.
3. **Assemble and Load:** Click the **Assemble** or **Load** button to move the instructions into the simulated RAM.
4. **Run:** Open the **Input/Output Console** within the simulator.
5. **Test:** Type integers and press enter. Once you type 0, the console will display the total sum.

YouTube +4

For professional development directly on Ubuntu without a simulator, you would typically use the NASM Assembler or YASM along with standard Linux system calls (`sys_read` and `sys_write`) to handle keyboard input and screen output.